

Internal Company Knowledge Assistant

Detailed project explanation for the FastAPI, Streamlit, LangChain, Azure OpenAI, AWS, Langfuse, and RAGAS MVP.

Purpose	Explain the implemented MVP architecture, component responsibilities, data flows, AWS services, evaluation strategy, and deployment path.
Audience	Engineers, ML/LLM practitioners, cloud reviewers, and project assessors who need to understand how the assistant works.
Project state	Greenfield scaffold implemented in this workspace with backend, frontend, ingestion, evaluation, tests, Docker, and AWS deployment templates.
Primary outcome	A chat-based internal knowledge assistant that retrieves grounded company answers from S3-indexed documents and records observability/evaluation metadata.
MVP lens	The project is designed to be credible in five days: containerized services, secure secret loading, RAG, agent tools, persistent history, Langfuse tracing, RAGAS evals, stress testing, and an ECS deployment path.

Contents

1. Executive overview
2. Architecture and request flow
3. Backend and frontend explanation
4. Agentic AI and RAG design
5. Security, secrets, and chat history
6. Observability, prompt versioning, and evals
7. AWS deployment model
8. Repository map, verification, and next steps

1. Executive Overview

The Internal Company Knowledge Assistant is a containerized application that lets employees ask questions against company knowledge. It combines a Streamlit chat interface, a FastAPI backend, a LangChain agent, Azure OpenAI, AWS storage and hosting services, Langfuse observability, and RAGAS evaluation.

The assistant is intentionally built as an MVP with production-shaped boundaries. The frontend stays thin and user-focused. The backend owns authentication, secret loading, retrieval, tool execution, model calls, chat persistence, and tracing. AWS services provide the operational foundation: S3 for documents, OpenSearch Serverless for vector search, DynamoDB for chat history, ECR/ECS for containers, CloudWatch for logs, and Secrets Manager for every secret value.

- Users must log in before they can ask questions.
- Questions are answered through a chat-only interface.
- The agent has at least three tools: RAG search, document catalog lookup, and CSV/table lookup.
- Answers return sources, tools used, input/output token estimates, latency, and Langfuse trace IDs.
- Chat history persists and is passed back into the agent so follow-up questions have context.
- Golden-data evals and a 100-query stress test provide measurable quality checks.

2. Architecture And Request Flow

The application is split into a frontend ECS service and a backend ECS service. The Streamlit service handles user interaction. The FastAPI service handles trusted operations and talks to AWS, Azure OpenAI, Langfuse, and the vector store.

```
User -> ALB -> Streamlit ECS Service -> FastAPI ECS Service
FastAPI -> Auth + AWS Secrets Manager
FastAPI -> DynamoDB Chat History
FastAPI -> LangChain Agent
Agent -> RAG Search Tool -> OpenSearch Serverless
Agent -> Document Catalog Tool -> S3 Manifest
Agent -> CSV/Table Lookup Tool -> S3 CSV Files
Agent -> Azure OpenAI Chat + Embeddings
FastAPI -> Langfuse Traces + Prompt Versions
```

Layer	Responsibility	Main services / files
User interface	Login, chat input, response display, session list, metadata panel.	Streamlit; frontend/streamlit_app.py
API layer	Validates tokens, exposes chat/session/document endpoints, centralizes service wiring.	FastAPI; backend/app/main.py
Agent layer	Builds context from tools and chat history, calls Azure OpenAI via LangChain.	backend/app/agent.py
Knowledge layer	Reads S3 manifest, searches vectors, looks up CSV rows.	S3, OpenSearch Serverless; storage.py, retrieval.py
State layer	Stores sessions and messages for persistent contextual conversations.	DynamoDB; history.py
Observability/evals	Prompt versions, traces, token metadata, RAGAS reports, stress reports.	Langfuse, RAGAS; observability.py, evals/

How One Chat Turn Works

- 1 The user signs in through Streamlit, which calls POST /auth/login.
- 2 FastAPI validates the credentials against password hashes loaded from AWS Secrets Manager.
- 3 Streamlit stores the bearer token in session state and sends it with chat requests.
- 4 FastAPI validates the token, identifies the user, and loads prior messages for the session.
- 5 The agent runs RAG search, document catalog lookup, and CSV/table lookup.
- 6 The agent builds a prompt with system instructions, chat history, tool context, and the user query.
- 7 Azure OpenAI generates the response through the LangChain wrapper.
- 8 The backend stores the user and assistant turns with sources, tools, token counts, latency, and trace ID.
- 9 Streamlit displays the answer and response details.

3. Backend Explanation

The FastAPI backend is the trusted orchestration layer. It keeps secrets and model access away from the browser, enforces authentication, coordinates retrieval and agent calls, and records history and response metadata.

Configuration	backend/app/config.py reads non-secret runtime values such as region, secret names, bucket, index, table, and prompt label.
Secrets	backend/app/secrets.py loads JSON secrets from AWS Secrets Manager. Environment variables contain only secret names and non-sensitive config.
Authentication	backend/app/auth.py verifies PBKDF2 password hashes and issues signed HMAC bearer tokens.
API routes	backend/app/main.py exposes /health, /auth/login, /chat, /chat/sessions, /chat/sessions/{id}, and /documents.
Agent	backend/app/agent.py loads history, runs tools, builds model context, calls Azure OpenAI through LangChain, and returns structured metadata.
Retries	backend/app/retries.py adds retry behavior for remote-facing paths such as Secrets Manager, S3, OpenSearch, and ingestion.

API Shape

```
POST /auth/login
{ "username": "user", "password": "password" }

POST /chat
{ "query": "What is our leave policy?", "session_id": "optional-existing-session-id" }

Response
{
  "session_id": "abc123",
  "answer": "Employees are entitled to...",
  "sources": [],
  "tools_used": ["rag_search"],
  "input_tokens": 1200,
  "output_tokens": 300,
  "latency_ms": 2400,
  "trace_id": "langfuse-trace-id"
}
```

4. Frontend Explanation

The frontend is a Streamlit app designed around the chat workflow. It starts with a login screen, then switches to the authenticated chat experience. The sidebar supports new chats, sign out, and previous chat sessions. Assistant messages include a collapsible metadata panel for debugging and demo evidence.

- The frontend never stores long-lived secrets.
- The bearer token stays in Streamlit session state for the active browser session.
- Previous sessions are loaded from FastAPI and can be reopened.
- Response metadata makes the system demonstrable: sources, tools, tokens, latency, and trace ID are visible.

5. Agentic AI And Tool Design

The project uses an agentic design rather than a single prompt-only chain. The backend registers tools with clear responsibilities, then the agent decides how to use available context. This creates a more convincing assistant because the system can combine semantic retrieval, document awareness, and exact structured lookup.

Tool	Purpose	Best suited for
rag_search	Semantic retrieval over indexed company document chunks with citation metadata.	Policy, process, handbook, FAQ, and procedural questions.
document_catalog	Lists and filters documents from the S3 manifest.	Questions about what knowledge exists, document type, department, or ownership.
table_lookup	Searches CSV-style records in S3 for exact values.	Structured facts such as contacts, escalation rows, owners, codes, or tabular policy values.

Why this is agentic

The system exposes multiple tools to the model-facing orchestration layer. RAG is one tool, not the whole assistant. The catalog and table tools improve factual coverage and make the system more robust for internal knowledge.

6. RAG And Ingestion Flow

Documents live in S3 under a raw prefix. The ingestion job reads supported files, extracts text, chunks content, creates embeddings through Azure OpenAI, indexes chunks into OpenSearch Serverless, and writes a manifest to S3. The manifest powers the document catalog and /documents endpoint.

- 1 Upload PDFs, markdown, text files, and CSVs to the configured S3 raw prefix.
- 2 Run the ingestion CLI from the backend container.
- 3 Parse source files and compute checksums.
- 4 Chunk text using LangChain text splitters, with a deterministic fallback splitter.
- 5 Generate embeddings with the Azure OpenAI embedding deployment.
- 6 Index each chunk into OpenSearch Serverless with metadata and source URI.
- 7 Write a JSON manifest back to S3 for document discovery.

Indexed field	Why it exists
key	Original S3 object key.
title	Human-readable filename/title.
uri	Citation path, for example s3://bucket/raw/policy.md.
text	Chunk text used by retrieval and answer grounding.
embedding	Vector representation for semantic search.
content_type	Differentiates PDF, text, markdown, and CSV content.
chunk_index	Preserves chunk order within a document.
checksum	Supports duplicate/change detection.
metadata	Extensible department, owner, document type, or date values.

7. Chat History And Context

Chat history is stored per user and session. On every turn, the backend loads prior messages and builds a bounded history context. This lets the assistant answer follow-up questions such as 'What about contractors?' or 'Can you summarize that?' without losing the thread.

Local mode	Uses an in-memory repository for development and tests.
Production mode	Uses DynamoDB with user_id as the partition key and sort_key for session/message records.
Context control	MAX_HISTORY_CHARS limits how much history is injected into the agent prompt.
Long sessions	Older messages are omitted with a summary marker once the history window becomes too large.

8. Security And Secrets

The project explicitly treats AWS Secrets Manager as the only source for secret values. The code avoids putting secrets into .env files, Docker images, source code, README commands, or Streamlit secrets. Environment variables contain secret names and ordinary configuration only.

Secret name	Contains	Used by
/company-assistant/{stage}/app	session_secret and auth_users password-hash map.	FastAPI authentication service.
/company-assistant/{stage}/azure-openai	endpoint, api_key, api_version, chat deployment, embedding deployment.	LangChain chat and embedding clients.
/company-assistant/{stage}/langfuse	public key, secret key, base URL.	Langfuse tracing and prompt management.

Security note

The MVP uses simple login because it is achievable in the five-day scope. For production, the natural upgrade is Cognito, SSO/SAML, or the company's existing identity provider.

9. Observability And Prompt Versioning

Langfuse is used for LLM observability and prompt governance. CloudWatch captures container logs, while Langfuse captures model traces, tool calls, prompt versions, token usage, latency, and failures. The backend returns trace IDs to the UI so a demo can move from an answer directly into observability evidence.

Tracing	Records model calls, retrieval/tool behavior, latency, token usage, and trace IDs.
Prompt versions	The system prompt can be loaded from Langfuse using dev or production labels.
Fallback prompt	If Langfuse prompt retrieval fails, the backend uses a safe default system prompt.
Demo value	The trace ID links the UI response to backend behavior and model/tool execution details.

10. Evaluation And Stress Testing

Evaluation is designed at two levels. Golden-data evaluation checks answer quality against expected answers and sources. Stress testing checks consistency and reliability across repeated paraphrases.

Evaluation	Implementation	Measures
Golden dataset	evals/golden_dataset.csv	Question, expected answer, expected source, tags.
RAGAS runner	evals/run_ragas_eval.py	Faithfulness, answer relevancy, context precision, context recall, plus fallback overlap scoring.
Stress test	evals/stress_test.py	100 paraphrased queries, latency, failures, source overlap, answer similarity.

Evaluation	Implementation	Measures
Unit tests	tests/	Auth, token validation, history persistence, bounded context, agent tool contract.

11. AWS Deployment Model

The target deployment is ECS Fargate. Backend and frontend images are built independently, pushed to ECR, and deployed as separate ECS services. An Application Load Balancer exposes the user interface and can route API traffic to the backend service. IAM task roles grant least-privilege access to the exact AWS resources needed.

AWS service	Role in the project	Important configuration
S3	Stores raw company documents and the ingestion manifest.	Bucket policy, raw prefix, manifest key, encryption.
ECR	Stores backend and frontend Docker images.	Separate repositories and immutable tags for releases.
ECS Fargate	Runs FastAPI and Streamlit containers.	Task roles, service networking, CPU/memory, health checks.
ALB	Provides browser access to the app.	HTTPS listener, security groups, optional IP/VPN restriction.
Secrets Manager	Stores all secret values.	Least-privilege GetSecretValue permissions.
DynamoDB	Persists chat sessions and messages.	PAY_PER_REQUEST table with user_id/sort_key keys.
OpenSearch Serverless	Stores the vector index for RAG search.	AOSS permissions, vector mapping, collection endpoint.
CloudWatch	Captures ECS task logs.	Backend and frontend log groups.

12. Repository Map

Path	Explanation
backend/app/main.py	FastAPI endpoint, dependency wiring, auth enforcement, and API route definitions.
backend/app/agent.py	Agent orchestration, context assembly, Azure OpenAI call, token estimates, and response metadata.
backend/app/auth.py	Password hashing, verification, and signed bearer-token creation/validation.
backend/app/secrets.py	AWS Secrets Manager JSON loading for app, Azure OpenAI, and Langfuse secrets.
backend/app/history.py	In-memory and DynamoDB chat history repositories.
backend/app/ingest.py	S3 document parsing, chunking, embedding, OpenSearch indexing, and manifest writing.
frontend/streamlit_app.py	Login and chat interface, session sidebar, response details panel.
evals/	Golden dataset, RAGAS runner, and 100-query stress-test runner.
infra/	ECS task definitions, IAM policy, DynamoDB schema, and OpenSearch index mapping.
tests/	Unit tests for auth, chat history, and the agent/tool contract.

13. Verification Completed

The scaffold was verified locally without live cloud credentials. These checks prove the project structure, core behavior, and deployment templates are internally consistent.

Python syntax	compileall passed for backend, frontend, eval scripts, and tests.
Unit tests	6 tests passed: authentication, token validation, password hashing, history persistence, bounded context, and agent tool contract.
Docker Compose	docker compose config produced a valid service configuration.
Infra JSON	ECS task definitions, IAM policy, DynamoDB table definition, and OpenSearch index mapping parsed successfully.

14. What Needs Live AWS/Azure Setup

- Create AWS Secrets Manager entries with real Azure OpenAI and Langfuse credentials.
- Create the S3 bucket and upload real company documents.
- Create the OpenSearch Serverless collection and index with the correct vector dimension.
- Create the DynamoDB chat history table.
- Build and push Docker images to ECR.
- Deploy ECS Fargate services and configure the Application Load Balancer.
- Run ingestion, RAGAS evals, and the 100-query stress test against live infrastructure.

15. Recommended Next Improvements

Improvement	Why it matters	Priority
Cognito or SSO	Replaces simple login with enterprise identity and lifecycle management.	High
Terraform or CDK	Makes infrastructure repeatable and reviewable.	High
Admin ingestion UI	Lets authorized users upload and re-index documents without CLI access.	Medium
Source preview	Lets users inspect the exact retrieved snippets behind an answer.	Medium
Feedback buttons	Captures human quality signals for future eval datasets.	Medium
PII redaction	Reduces risk when indexing sensitive internal documents.	Medium
Cost dashboard	Tracks token spend, retrieval latency, and query volume.	Low

16. Final Summary

This project is a strong five-day MVP because it demonstrates the full production shape of an internal knowledge assistant: containerized services, AWS hosting path, secure secret management, agentic tool use, RAG, persistent context, observability, prompt versioning, evaluation, and stress testing. The scaffold can run locally with development settings, then move toward production once AWS resources and live Azure OpenAI/Langfuse secrets are configured.